

MPI Session WG

Current ideas: Spawning, Pset operations, Dictionaries

Working document

Session-friendly spawning

Feedback of June 2026 Forum meeting

- Relation of spawn operation, process sets, and session(s) was unclear
 - Need full example
- Non-blocking spawn (MPI_ISPAWN)
 - Keep as separate new procedure (no combination within MPI_SPAWN)
 - What does “completion” of a spawn request mean to MPI without communication establishment?
 - What does MPI know about the spawned processes?
 - Need “spawn completion levels” (similar to thread levels) via info parameter?

Other discoveries by the Session WG

- Sect. 11.3.2: “Process set names have a Uniform Resource Identifier (URI) format.”
- Scoping of process sets is not clear in MPI
 - Even without new spawn proposal!
 - How “unique” must process set names be?
 - Per session? Per process? Global?
- Separate topic!
 - Related to session-friendly spawn proposal
 - Will be addressed separately!

New to MPI: Spawn procedures

- MPI Spawn and MPI Ispawn
 - Start new (MPI) processes, no communication establishment
 - Parameters mostly resemble MPI_COMM_SPAWN_MULTIPLE
 - Collective over input comm
- Operation completes once all procs in comm have been notified about
 - successful completion of spawn operation in runtime system and defined new process set(s) - if any
 - or spawn error

```
MPI_SPAWN( int count, char **array_of_commands,  
           char ***array_of_argv,  
           const int *array_of_maxprocs,  
           MPI_Info *array_of_info,  
           int root,  
           MPI_Comm comm)
```

```
MPI_ISPAWN(int count, char **array_of_commands,  
           char ***array_of_argv,  
           const int *array_of_maxprocs,  
           MPI_Info *array_of_info,  
           int root,  
           MPI_Comm comm,  
           MPI_Request *request)
```

New to MPI process set management

- Remote process sets: Allow to "see" process sets in which a process is not a member
 - MPI_GROUP_RANK for group derived from remote process set yields MPI_UNDEFINED
- Allow empty process sets
 - Attribute "mpi_size" == 0
 - Group created from empty process set: MPI_GROUP_EMPTY
- Reserved info key "pset_name" on MPI_(I)SPAWN
 - Define a process set name for the new processes (per app)
 - Process set(s) known in processes involved in comm once the operation is complete
 - Process set(s) retrievable via MPI_SESSION_GET_NTH_PSET etc. as remote pset(s)
- Standard MPI pset "mpi://PARENT"
 - Pre-defined process set (similar to "mpi://SELF" and "mpi://WORLD")
 - Processes that are not spawned via MPI_(I)SPAWN: Yields empty group
 - Processes that are spawned via MPI_(I)SPAWN: Yields group of comm used in MPI_(I)SPAWN on spawner side

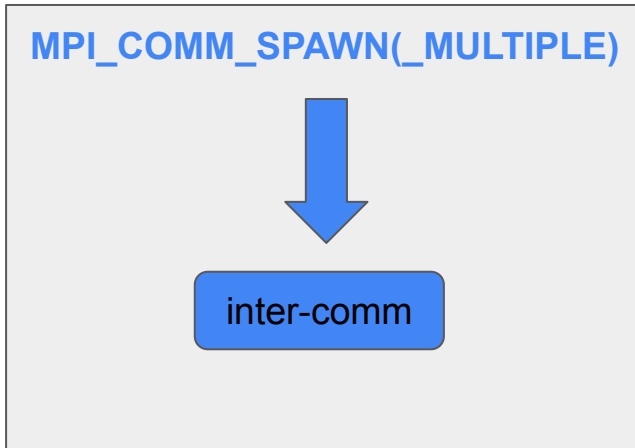
Options for spawning with MPI

#	Option	MPI procedures used by spawner(s)	MPI procedures used by spawnee(s)	MPI comm	MPI Session	MPI World
1	Legacy	MPI_COMM Spawn(_MULTIPLE)	MPI_INIT, MPI_COMM_GET_PARENT	Inter-comm	NO	YES
2	Dissect	MPI_(I)SPAWN, MPI_PUBLISH_NAME, MPI_COMM_ACCEPT	MPI_LOOKUP_NAME, MPI_COMM_CONNECT	Inter-comm	YES	YES
3	Session inter-comm	MPI_(I)SPAWN, pset_name in info, MPI_GROUP_FROM_SESSION_PSET, MPI_INTERCOMM_CREATE_FROM_GROUPS	MPI_GROUP_FROM_SESSION_PSET using pset "mpi://PARENT", MPI_INTERCOMM_CREATE_FROM_GROUPS	Inter-comm	YES	NO
4	Session intra-comm	MPI_(I)SPAWN, pset_name in info, MPI_GROUP_FROM_SESSION_PSET, MPI_GROUP_UNION, MPI_COMM_CREATE_FROM_GROUP	MPI_GROUP_FROM_SESSION_PSET using pset "mpi://PARENT", MPI_GROUP_UNION, MPI_COMM_CREATE_FROM_GROUP	Intra-comm	YES	NO

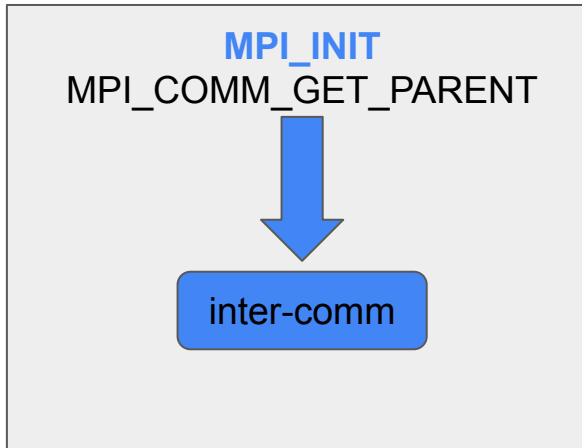
Option 1: Legacy spawn

#	Option	MPI procedures used by spawner(s)	MPI procedures used by spawnee(s)	MPI comm	MPI Session	MPI World
1	Legacy	MPI_COMM_SPAWN(_MULTIPLE)	MPI_INIT, MPI_COMM_GET_PARENT	Inter-comm	NO	YES

Spawner(s)



Spawnee(s)



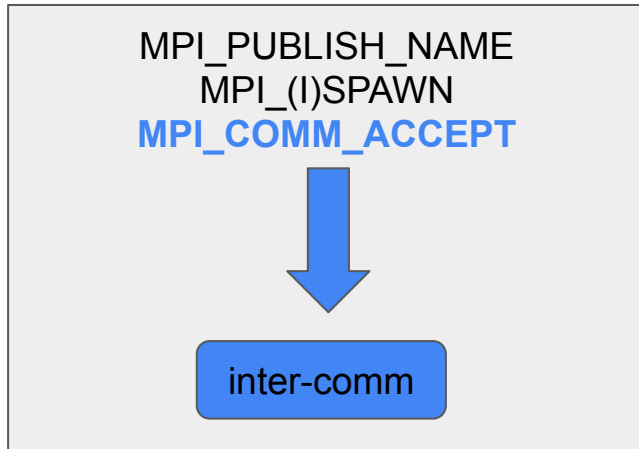
Runtime tasks
(besides the
spawn)

Exchange spawners' leader port with spawnees' leader
(KVS or environment var)

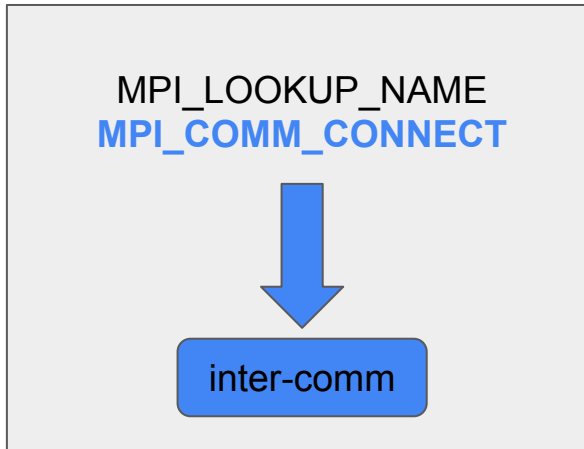
Option 2: Dissect spawn

#	Option	MPI procedures used by spawner(s)	MPI procedures used by spawnee(s)	MPI comm	MPI Session	MPI World
2	Dissect	MPI_(I)SPAWN, MPI_PUBLISH_NAME, MPI_COMM_ACCEPT	MPI_LOOKUP_NAME, MPI_COMM_CONNECT	Inter-comm	YES	YES

Spawner(s)



Spawnee(s)



Runtime tasks
(besides the
spawn)

Support MPI_PUBLISH_NAME and MPI_LOOKUP_NAME

Option 3: Session inter-comm

#	Option	MPI procedures used by spawner(s)	MPI procedures used by spawnee(s)	MPI comm	MPI Session	MPI World
3	Session inter-comm	MPI_I(1)SPAWN, pset_name in info, MPI_GROUP_FROM_SESSION_PSET, MPI_INTERCOMM_CREATE_FROM_GROUPS	MPI_GROUP_FROM_SESSION_PSET using pset "mpi://PARENT", MPI_INTERCOMM_CREATE_FROM_GROUPS	Inter-comm	YES	NO

Spawner(s)

```
MPI_INFO_SET(&info, "pset_name", "mypset")
MPI_I(1)SPAWN(..., info, comm)
MPI_GROUP_FROM_SESSION_PSET
(..., "mypset", &remote_group)
MPI_INTERCOMM_CREATE_FROM_GROUPS
(local_group, ..., remote_group, .., &intercomm)
```



inter-comm

Spawnee(s)

```
MPI_GROUP_FROM_SESSION_PSET
(..., "mpi://PARENT", &parent_group)
MPI_INTERCOMM_CREATE_FROM_GROUPS
(local_group, ..., parent_group, ..., &intercomm)
```



inter-comm

Runtime tasks
(besides the
spawn)

Populate pset "mypset" on spawners' side in procs of comm (if not done using comm)
Exchange member of pset "mpi://PARENT" (procs in comm) from spawners to spawnees
Exchange port information between local and remote leaders for inter-comm creation

Option 3: Session intra-comm

#	Option	MPI procedures used by spawner(s)	MPI procedures used by spawnee(s)	MPI comm	MPI Session	MPI World
4	Session intra-comm	MPI_I(1)SPAWN, pset_name in info, MPI_GROUP_FROM_SESSION_PSET, MPI_GROUP_UNION, MPI_COMM_CREATE_FROM_GROUP	MPI_GROUP_FROM_SESSION_PSET using pset "mpi://PARENT", MPI_GROUP_UNION, MPI_COMM_CREATE_FROM_GROUP	Intra-comm	YES	NO

Spawner(s)

```
MPI_INFO_SET(&info, "pset_name", "mypset")
MPI_I(1)SPAWN(..., info, comm)
MPI_GROUP_FROM_SESSION_PSET
(..., "mypset", &remote_group)
MPI_GROUP_UNION
(local_group, remote_group, &newgroup)
MPI_COMM_CREATE_FROM_GROUP
(newgroup,..., &newcomm)
```



intra-comm

Spawnee(s)

```
MPI_GROUP_FROM_SESSION_PSET
(..., "mpi://PARENT", &parent_group)
MPI_GROUP_UNION
(parent_group, local_group, &newgroup)
MPI_COMM_CREATE_FROM_GROUP
(newgroup, ..., &newcomm)
```



intra-comm

Runtime tasks
(besides the
spawn)

Populate pset "mypset" on spawners' side in procs of comm (if not done using comm)
Exchange member of pset "mpi://PARENT" (procs in comm) from spawners to spawnees
Exchange connection infos for procs in newgroup (similar to startup bootstrapping)

MPI Dictionaries

New to MPI: Dictionary procedures

`MPI_DICT_CREATE(char *dictname, MPI_Info info)` -> create a new, named dictionary

`MPI_DICT_REMOVE(dictname)` -> remove an existing, named dictionary

`MPI_DICT_DATA_PUBLISH(dictname, keys_values, info)` -> publish key-values in existing dictionary

`MPI_DICT_DATA_LOOKUP(dictname, keys_values, info)` -> lookup key-values in existing dictionary

`MPI_DICT_DATA_UNPUBLISH(dictname, keys)` -> unpublish key-values in existing dictionary

Objective:

- Enable asynchronous data exchange without direct communication channels
- Use case: Coordination of dynamic processes
- E.g. publish a PSet name

Generalization of MPI_Publish/Lookup_name

```
int MPI_Publish_name(const char *service_name, MPI_Info info, const char *port_name)
```

```
int MPI_Lookup_name(const char *service_name, MPI_Info info, char *port_name)
```

Main differences:

- Multiple named dictionaries instead of only one, global dictionary
- General purpose, not only port info

Scope:

- Implementation-defined
- High-Quality implementation: influence scope via info objec

MPI PSet Operations

New to MPI: PSet Operation procedures

MPI_SESSION_IPSETOP(session, pset_op, input_psets, col_info, output_psets, output_size, request)

- Objectives:
 - Express dynamic process structure of applications via PSetOps
 - UNION, DIFFERENCE, INTERSECTION, ADD, SUB, ...
 - Couple dynamic process management with resource allocation hints via COL object (used by RM for decisions)
- Resource Manager:
 - The RM decides on PSetOps and corresponding resource allocations.
 - There is no timing guarantee for RM decisions
- PSet assumptions:
 - Remote psets: Allow to "see" psets where a process is not a member
 - Property of pset: "is_member"
 - Property of pset: "dynamic": pset was created by a PSetOp
 - Property of pset: "primary": one proc of the pset is primary proc
 - **Upon return, output PSets are available to all procs -> scoping?**
 - Processes are "spawned" by the runtime after receiving response from RM
 - Once processes are running, PSetOp returns, providing the output PSet names (RM assigned)